



TEAM SHARING SESSION

# QA AI Skills Repo

ทำความเข้าใจ · ใช้งาน · ปรับแต่ง · contribute

v1.0 · 8 พฤษภาคม 2569 · Tester Lead Team

# Agenda · 2 ชั่วโมง (Lecture 60 + Workshop 60)

## Part 1 · Lecture + Demo · 60 นาที

- (10') Skill คืออะไร · โครงสร้าง · Frontmatter
- (10') จุดประสงค์ของ repo · ปัญหาที่แก้ · 14 Skills
- (20') Deep dive 5 skills ที่ใช้บ่อย
- (10') qa-standards · Severity/Priority/Sizing
- (5') AI Guardrails 5 ข้อ + ความคาดหวัง
- (5') Live Demo

## ☕ Break · 10 นาที

## Part 2 · Workshop · 50 นาที

- (5') Lab 0 — Install plugin
- (5') Lab 1 — รับ bug-report-writer
- (10') Lab 2 — เขียน TC จาก SRS + tuning
- (5') Lab 3 — สร้าง project-context.md
- (10') Lab 4 — แก้ skill + validate + PR
- (15') Free Play — ลองใช้ skill อื่นๆ

## 🎯 Wrap-up + Q&A · 10 นาที

เป้าหมาย: จบ session แล้วทุกคน (1) ติดตั้ง plugin ได้ (2) รับ skill เป็น (3) รู้วิธี tuning + เปิด PR ตัวแรก

# Skill ของ Claude คืออะไร

**Skill** = ชุด instruction (Markdown) + template + reference ที่บอก Claude ว่า “งานประเภทนี้ทำยังไง” — โหลดเข้า context **เฉพาะตอนถูก trigger**

## จุดเด่น

- Trigger ผ่าน **slash command** (เช่น /test-case-writer) หรือ **natural-language keyword** (เช่น “เขียน test case จาก SRS นี้”)
- ไม่ต้อง prompt engineering ใหม่ทุกครั้ง — มาตรฐานเดียวกันทั้งทีม
- แฮร์เป็น **plugin** — ติดตั้งครั้งเดียวใช้ได้ทุก skill
- แก่ที่ Markdown ไฟล์เดียว ทีมได้พร้อมกัน

## 🎯 ต่างจาก ChatGPT/Prompt กว้างไปยิ่ง

**Prompt กว้าง:** ต่างคนต่างพิมพ์ พลัฟร์คนละมาตรฐาน

**Custom GPT:** ผูกกับ account คนสร้าง แฮร์ยาก แก้ไม่เห็น diff

**Claude Skill:** อยู่ใน git repo, version control, code review ได้, ทุกคนใช้ตัวเดียวกัน

Skills ที่ใช้ได้ทันที

# 14

# โครงสร้าง 1 Skill · มีอะไรบ้าง

## Folder structure

```
skills/<skill-name>/
├── SKILL.md           ← instruction หลัก
├── templates/        ← output template
│   ├── *-th.md
│   └── *.csv
├── references/       ← reference เฉพาะ skill
└── examples/         ← input → output ตัวอย่าง
```

Claude อ่าน **frontmatter** ของ SKILL.md ก่อน เพื่อรู้ว่าเมื่อไรต้อง trigger — ค่อย load body ทั้งหมดเข้า context ตอน trigger จริง

## SKILL.md · 8 sections มาตรฐาน

1. **Frontmatter** — name + description + trigger
2. **Purpose** — เป้าหมาย + effort savings
3. **When to Use** — SDP mapping
4. **Inputs** — สิ่งที่ต้องเตรียม
5. **Outputs** — format + template
6. **Process** — step-by-step
7. **Quality Gate** — checklist ก่อนส่ง
8. **AI Guardrails** — ข้อควรระวัง
9. **Chain** — เชื่อมกับ skill อื่น

ดูเต็ม: SKILL-TEMPLATE.md

# Frontmatter · หัวใจของ Skill

ส่วน metadata ที่ Claude อ่านเพื่อรู้ว่า “skill นี้คืออะไร · ตอนไหนต้องเรียก”

---

**name:** test-case-writer

**description:** เขียน test case จาก requirement document (SRS/PRD/spec/user story)

ให้ครอบคลุมและอ่านง่าย – รองรับ SIT mode (technical view, 23 cols), UAT mode

+ testing techniques (ECP, BVA, Decision Table, State Transition).

Trigger เมื่อ user ส่ง requirement file/SRS/PRD/spec/user story และขอให้เขียน

test case, test scenario, SIT test case, UAT test case, "write test cases",

"create test scenarios", "generate UAT checklist".

Maps to SDP §5.3.1 (Process 2, 6).

---

## name

kebab-case · ตรงกับชื่อโฟลเดอร์ · unique ทั้ง repo

## description

ทำอะไร + input → output + trigger keywords (TH+EN) + SDP mapping

## trigger

Claude scan keyword ใน user message ถ้า match → load skill เข้า context

# | จุดประสงค์ของ Repo นี้

รวม Claude Skills สำหรับทีม Tester — ทุกคนใช้ตัวเดียวกัน  
ผลลัพธ์มาตรฐานเดียวกัน อัปเดตจุดเดียว

- 1 Standardize work product**  
TC, Bug Report, Test Plan, Report ใช้ template + Severity/Priority/Sizing เดียวกันทุกคน — alignment กับ SDP §5
- 2 ลด effort งานเอกสาร ~50%**  
AI draft 70-80% เราเหลือแค่ review/verify — เอะเวลาไปทำงานที่ AI ทำแทนไม่ได้
- 3 Knowledge as code**  
วิธีเขียน TC ที่ดี / วิธีรีวิว / วิธีรายงานผล อยู่ใน git — review ได้ versioning ได้ rollback ได้
- 4 Onboarding Tester ใหม่เร็วขึ้น**  
คนใหม่อ่าน skill + ลองรันได้เลย ไม่ต้องนั่งสอน prompt 1-1

Skills ที่ใช้ได้

14

SDP processes ที่ครอบคลุม

12

เป้าลด effort/artifact

~50%

Workflow chain (SIT/UAT/Perf)

3

# ทำเพื่ออะไร · แก้ปัญหาอะไร



**Pain · เขียน TC ใช้เวลาเยอะ — 50-60% ของ sprint หมวดเอกสาร**

✓ test-case-writer draft ให้ 80% แล้วย verify อีก 20%



**Pain · Rework รอบใหญ่ตอน PM บอก "เข้าใจผิด"**

✓ requirement-analyzer เช็ค Readiness + ส่ง PM confirm ก่อนเขียน TC



**Pain · สรุป Test Report ตอน sprint จบ ใช้เวลาทั้งวัน**

✓ test-report-writer สรุปจาก Jira export ได้ใน 10 นาที



**Pain · TC แต่ละคนเขียนคนละ format / Severity ใช้คนละ scale**

✓ qa-standards.md บังคับใช้ทุก skill — มาตรฐานเดียว



**Pain · Bug Report ใน Jira ไม่ครบ field ทีม Dev reproduce ไม่ได้**

✓ bug-report-writer ใส่ครบ — repro steps + env + Severity×Priority



**Pain · Tester ใหม่ onboarding 2 อาทิตย์ยังเขียน TC ไม่ได้คุณภาพทีม**

✓ skill ทำหน้าที่ "senior Tester ช่างๆ" — สอน + draft ไปพร้อมกัน

# Before vs After · เห็นภาพชัด

## ❌ Before — ไม่มี Skill

- PM ส่ง BRD มา → Tester เปิด Word เริ่มเขียน TC จากศูนย์
- แต่ละคน prompt ChatGPT ต่างกัน ผลคนละแบบ
- Severity 1-5 หรือ Critical/High/Medium? คนละความเข้าใจ
- Bug Report บางใบไม่มี repro step ต้องไล่ถาม
- Test Report ตอน UAT จบ — ใช้เวลา 1 วัน copy-paste
- Tester ใหม่: "พี่ TC แบบนี้โอเคมั้ย?" ทุกครั้ง
- knowledge อยู่ในหัวคน — ลาออก = หาย

## ✅ After — มี Skill

- /requirement-analyzer ใช้ BRD พร้อมก่อน → PM confirm
- /test-case-writer draft TC ตาม template เดียวกัน
- Severity = Critical/Major/Minor/Trivial — บังคับใน qa-standards
- /bug-report-writer field คสบทุกใบ → Dev reproduce ได้ทันที
- /test-report-writer สรุปรายการ Jira export → 10 นาที
- Tester ใหม่: รับ skill เห็น output มาตรฐานทีมเลย
- knowledge อยู่ใน git — ธีวได้ versioning ได้



# ภาพรวม 14 Skills · 4 กลุ่ม

## PRE-TESTING

requirement-analyzer

data-type-matrix-generator

## TESTING PROCESS

· ครอบคลุม 12 SDP

test-plan-writer

test-case-writer

test-case-reviewer

test-report-writer

perf-test-generator

perf-result-analyzer

## SUPPORTING

test-matrix-generator

bug-report-writer

robot-test-generator

e2e-test-generator

## LEAD UTILITY

weekly-update-writer

handoff-writer

💡 6 skills ครอบคลุม 12 SDP processes — เพราะ skill เดียวใช้ซ้ำหลาย mode (SIT/UAT/Perf)

# Workflow Chain · ใช้งานจริง

## SIT Chain

BRD/PRD

- ↓ requirement-analyzer
- ↓ test-plan-writer
- ↓ test-case-writer
- ↓ test-case-reviewer
- ↓ [Execute]
- ↓ bug-report-writer
- ↓ test-report-writer

## UAT Chain

SIT TC approved

- ↓ test-case-writer (uat)
- ↓ test-case-reviewer (uat)
- ↓ test-plan-writer (uat)
- ↓ [Execute by User]
- ↓ test-report-writer (uat)
- ↓ User Sign-off
- ↓ → Production

## Perf Chain

NFR + API Spec

- ↓ test-plan-writer (perf)
- ↓ perf-test-generator
- ↓ [Run Load/Stress]
- ↓ perf-result-analyzer
- ↓ test-report-writer (perf)
- ↓ Tuning Recommend

 **พนักงาน:** skill เดียวกันใช้ได้ 3 mode — test-plan-writer รับ argument mode=sit/uat/perf

# | ความคาดหวัง 5 ข้อ ก่อนเริ่มใช้

- 1 AI ช่วย draft, Tester review**  
AI draft 70-80% เราต้อง review/approve อีก 20-30% ก่อนส่ง · AI ไม่ sign-off แทนคน
- 2 Cross-check กับ source**  
ทุก output ต้องตรวจสอบกับ SRS/PRD/Jira ผ่าน Traceability Matrix · AI hallucinate ได้
- 3 ห้าม commit sensitive data**  
PII / password / token / production credential · ใช้ [REDACTED] หรือ dummy data
- 4 Expected Result วัดได้**  
ห้าม “ทำงานถูกต้อง” / “แสดงผลปกติ” · ต้องระบุตัวเลข สถานะ ค่าเฉพาะ
- 5 ห้าม make up number**  
ถ้า AI ไม่มีข้อมูลจริง (NFR/SLA/business rule เฉพาะลูกค้า) ให้ระบุ “TBD” หรือถามเรา · ห้ามเดา

# | งานที่ AI ทำแทนไม่ได้ · ของเรา 100%

AI ลด effort 50% แต่ 6 งานนี้ต้องเป็นคนเท่านั้น

## Cross-check Business Rule เฉพาะลูกค้า

promotion / VIP benefit / business logic ที่ AI ไม่รู้

## Verify Environment + Credential จริง

AI ใด IP/URL/DB connection จริงไม่ได้

## Approve Deferred Bug

ต้องเป็น PM / Stakeholder decision

## Accept / Reject Coverage Gap

บาง gap ยอมรับได้ (Phase 2) บาง gap ต้องแก้ทันที

## Sign-off Ready / Not Ready

ความรับผิดชอบทางวิชาชีพ ลายเซ็นเป็นของคน

## Communicate กับ PM/BA/Dev

การ negotiate / ขอเลื่อน timeline · human-to-human

DEEP DIVE

# 5 Skills ที่ใช้บ่อย

Input · Output · Sample Prompt · When to use

requirement-analyzer

test-case-writer

test-case-reviewer

bug-report-writer

test-report-writer



# 1. requirement-analyzer

เมื่อไหร่ใช้: ก่อนเขียน SIT TC ทุกครั้ง · เช็คว่า BRD/PRD พร้อมให้ AI ทำงานหรือยัง · ป้องกัน rework รอบใหญ่

## INPUT

1. BRD / PRD / SRS / User Story
2. Module/Feature ที่จะวิเคราะห์
3. Project + PM ที่ต้อง confirm

## OUTPUT

1. Readiness Score (Ready / Needs-clarification / Not-ready)
2. Normalized Requirement (FR ID + 9 fields)
3. PM/BA Confirmation Doc (ส่ง review)
4. List of Open Questions

 พิมพ์แบบนี้ให้ AI:

@PRD-loyalty.md วิเคราะห์ requirement – module: Loyalty, project: PEA  
ส่ง PM (คุณสมศรี) review ก่อนเขียน TC

## 2. test-case-writer

เมื่อไหร่ใช้: คลัง requirement-analyzer ผ่าน + PM confirm · คสอบ SIT (technical) และ UAT (business)

### INPUT

1. SRS / Normalized Requirement
2. Module ID + project-context.md
3. Mode: sit / uat
4. Testing technique (ECP / BVA / Decision Table / State Transition)

### OUTPUT

1. Test Cases 23-column (TC ID, Steps, Expected, Priority, Severity, Sizing)
2. Traceability Matrix (FR ↔ TC)
3. Sizing Summary Block (S/M/L/XL count)
4. รองรับ TH/EN + Markdown/CSV

 พิมพ์แบบนี้ให้ AI:

@normalized-req.md เขียน SIT TC – ภาษาไทย CSV  
ครอบคลุม ECP, BVA, Decision Table · เน้น negative + boundary

### 3. test-case-reviewer

เมื่อไหร่ใช้: หลังเขียน TC เสร็จ · peer review ก่อนเริ่ม execute · หา gap กับ SRS

#### INPUT

1. ไฟล์ Test Cases ที่จะ review
2. SRS / Normalized Requirement (compare)
3. Mode: sit / uat

#### OUTPUT

1. Review Report ตาราง: TC ID / ปัญหา / ระดับ Must Fix / Should Fix / ข้อเสนอแนะ
2. Coverage gap analysis (FR ที่ไม่มี TC ครอบคลุม)
3. Checklist 8 จุด: Expected ข้อ · Precondition ครอบคลุม · Positive/Negative/Boundary ครอบคลุม · Traceability ไม่มี gap

💬 พิมพ์แบบนี้ให้ AI:

review SIT test case @testcases\_sit\_login\_20260420.md  
เทียบ SRS @docs/srs-login.md - หา gap + ปัญหา



## 4. bug-report-writer

เมื่อไหร่ใช้: ทุกครั้งที่เจอ defect ระหว่าง execute · log เข้า Jira ตาม template มาตรฐาน

### INPUT

1. Symptom ที่เจอ + Steps to reproduce
2. Environment (URL/version/browser)
3. Expected vs Actual
4. Screenshot/log (optional)
5. TC ID ที่เกี่ยวข้อง

### OUTPUT

1. Bug Report พร้อม paste เข้า Jira/Linear/GitHub
2. Severity × Priority Action Label ตาม qa-standards
3. Reproduce steps คสม → Dev reproduce ได้ทันที
4. รองรับ TH / EN

### พิมพ์แบบนี้ให้ AI:

เจอ login ไม่ผ่าน Chrome 130 macOS staging – error 500  
TC: AUTH-TC-005 · severity: Major · เขียน bug report Jira

## 5. test-report-writer

เมื่อไหร่ใช้: หลัง execute เสร็จ · สรุปผลเทียบ Exit Criteria + AI Effort Savings KPI

### INPUT

1. Test Execution Data (Jira export / Excel / CSV)
2. Test Plan (สำหรับเทียบ Exit Criteria)
3. Mode: sit / uat / perf
4. Sign-off info (User name + วันที่)

### OUTPUT

1. Summary table (Total/Pass/Fail/Block/Not Run)
2. Exit Criteria Evaluation (Pass/Fail vs target)
3. Defect Summary by Severity + Status + Deferred List
4. AI Effort Savings section (Estimate vs Actual)
5. Conclusion + Recommendation

 พิมพ์แบบนี้ให้ AI:

สรุป SIT Report จาก @sit\_execution.csv  
เทียบ Exit Criteria ใน @sit\_plan\_leave\_20260420.md

# qa-standards.md

## 1. AI Guardrails 5 ข้อ

**Single source of truth** — ทุก skill อ่าน standard นี้  
ข้อมูลไหลจาก TC → Plan → Report ได้ไม่ต้องง้อ scale

# Severity Scale · 4 ระดับ (บังคับทุก skill)

ความรุนแรงของปัญหา — ระดับผลกระทบหาก TC fail

| Level                                                                                             | ความหมาย                            | ตัวอย่าง                             | SLA Fix (SIT) |
|---------------------------------------------------------------------------------------------------|-------------------------------------|--------------------------------------|---------------|
|  <b>Critical</b> | ระบบหลักพัง ใช้งานไม่ได้            | ระบบล่ม · ชำระเงินไม่ผ่าน            | ≤ 1 วัน       |
|  <b>Major</b>    | ฟังก์ชันหลักใช้ไม่ได้ ระบบยังรันได้ | Login พัง · ข้อมูลไม่บันทึก          | ≤ 2 วัน       |
|  <b>Minor</b>    | ปัญหาเล็กน้อย ไม่กระทบหลัก          | UI ไม่ตรง Figma · validation แจ้งผิด | ใน sprint     |
|  <b>Trivial</b>  | จุกจิก ไม่กระทบการใช้งาน            | Wording · alignment เพี้ยน           | best effort   |

💡 อ้างอิงจาก **Ayodia TEST DEFINITION template** — ใช้คำเดียวกันทั้งทีม ไม่ใช้ Sev1-4 อีกต่อไป

# Priority Scale · 4 ระดับ (บังคับทุก skill)

ความเร่งด่วนของการทดสอบ/แก้ไข — ดูจากความสำคัญต่อธุรกิจ + เวลา

| Level                                                                                             | ความหมาย                                | ตัวอย่าง                         |
|---------------------------------------------------------------------------------------------------|-----------------------------------------|----------------------------------|
|  <b>Critical</b> | ต้องแก้ทันที เพราะ Block งานอื่น        | ปุ่ม Submit พัง — ทดสอบต่อไม่ได้ |
|  <b>High</b>     | สำคัญต่อ Core Function · ต้องทำในรอบนี้ | Register ใช้งานไม่ได้            |
|  <b>Medium</b>   | สำคัญรองลงมา · ก็นรอบถัดไปได้           | การค้นหาช้ากว่าปกติ              |
|  <b>Low</b>      | ไม่เร่งด่วน ทำทีหลังก็ได้               | UI ไม่ตรง Figma · สปีดโตน        |

## ⚠ Priority ≠ Severity

**Trivial bug (typo)** อาจเป็น **Critical Priority** ได้ ถ้าลูกค้าใหญ่บ่น

→ ทั้งคู่ต้องระบุแยกกัน ใน Bug Report ทุกใบ

# Severity × Priority Matrix · Action Label

จับคู่ Severity กับ Priority แล้วได้ **Action Label** — บอกชัดว่าต้องจัดการยังไง

| Sev \ Pri                                                                                         |  <b>Critical</b> |  <b>High</b> |  <b>Medium</b> |  <b>Low</b> |
|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
|  <b>Critical</b> | <b>Blocker</b>                                                                                    | Urgent                                                                                        | Important                                                                                         | Deferred Critical                                                                              |
|  <b>Major</b>    | High Business Risk                                                                                | <b>Standard High</b>                                                                          | Manageable                                                                                        | Can Delay                                                                                      |
|  <b>Minor</b>    | Prioritize If Impacted                                                                            | Optional but Noted                                                                            | Acceptable Delay                                                                                  | Low Impact Cosmetic                                                                            |
|  <b>Trivial</b>  | Non-critical Visible                                                                              | Minor Fix Suggested                                                                           | Schedule Later                                                                                    | Optional                                                                                       |

💡 **bug-report-writer** ใส่ Action Label ให้อัตโนมัติ — ทีม dev/PM อ่านปุ๊บรู้ว่าต้องทำอะไร

# Test Sizing + Schedule Formula

## Sizing Scale (บังคับทุก TC)

| Size | Hours | Steps | ลักษณะ            |
|------|-------|-------|-------------------|
| S    | 0.17  | 1-3   | smoke / 1 field   |
| M    | 0.42  | 4-8   | form + ตรวจสอบ    |
| L    | 0.75  | 9-15  | multi-step + data |
| XL   | 1.25  | 15+   | E2E ข้าม role     |

**Midpoint** = ตัวเลขที่ test-plan-writer ใช้คำนวณ schedule

## Schedule Formula

$$\text{Total} = \text{Prep} + \text{Exec} \times 1.5 + \text{Review} + \text{Report} + \text{Buffer}$$
  
20%

- **Prep:** Total TC  $\times$  0.1 hr
- **Exec Cycle 1:**  $\Sigma$  Sizing midpoint
- **Review:** Total TC  $\times$  0.05 hr
- **Defect Fix + Retest:** Exec  $\times$  30%
- **Regression:** Exec  $\times$  20%
- **Report:** 4 hr fixed
- **Buffer:** (sum)  $\times$  20%

## Velocity baseline

6 productive hr/day  $\cdot$  10 days/sprint = **60 hr/tester/sprint**

# AI Guardrails · 5 ข้อ (อิง SDP §5.3.3)

หลักการ: AI = Draft & Assist · Tester = Review & Approve

- 1 AI Hallucinate Requirement**  
AI อาจสร้าง TC จาก requirement ที่ไม่มีในเอกสาร · **ป้องกัน:** Cross-check ทุก output กับ SRS/PRD ผ่าน Traceability Matrix
- 2 ไม่รู้ Business Context เฉพาะ**  
AI ไม่รู้ business rule เฉพาะลูกค้า (tax, discount logic) · **ป้องกัน:** Tester/BA เพิ่ม TC เฉพาะทางหลัง AI draft
- 3 ไม่รู้ Environment จริง**  
AI อาจระบุ Server/DB/URL ผิด · **ป้องกัน:** update environment ผ่าน project-context.md
- 4 อาจสรุปตัวเลขผิด**  
Generate Report จาก raw data → อาจนับ/รวมผิด · **ป้องกัน:** ตรวจสอบตัวเลขกับ source data ทุก row
- 5 ข้อมูล Sensitive ห้ามใส่ AI**  
ชื่อจริง · เลขบัตร · ข้อมูลการเงิน อาจรั่วไป LLM provider · **ป้องกัน:** [REDACTED] / dummy data / env var



# Guardrails · Do / Don't ที่เห็นบ่อย

## ❌ Expected Result กำกวม

"ระบบทำงานถูกต้อง"

### ✅ ใช้แบบนี้

แสดง toast 'บันทึกสำเร็จ' สีเขียว  
ภายใน 2 วิ → redirect /dashboard  
DB tbl\_leave.status='PENDING'

## ❌ AI เดาตัวเลข

p95: 250ms · TPS: 150  
(no source)

### ✅ ใช้แบบนี้

p95: TBD (ขอจาก architect)  
TPS: TBD

## ❌ commit Sensitive data

User: somsri@bank.co.th  
Pwd: P@ssw0rd2026!  
Account: 1234-5678-9012

### ✅ ใช้แบบนี้

User: [REDACTED]  
Pwd: [REDACTED]  
Account: 0000-0000-0000

## ❌ ส่ง AI output ตรงไป User

ส่งให้ stakeholder โดยไม่ผ่าน Tester review

### ✅ ใช้แบบนี้

AI draft → Tester review → fix gap → approve → ส่ง

# ดูของจริง

## Tester Lead จะ demo ให้ดู:

1. เปิด Claude Code · พิมพ์ /test-case-writer
2. แนน SRS file ตัวอย่าง · พิมพ์ prompt
3. ดู output ออกมาเป็น TC 23-column
4. ลองแก้ prompt ใ้ห้มัน negative case
5. เปรียบเทียบ output กับที่เขียนเอง

 ดูที่หน้าจอใหญ่ — ยังไม่ต้องเปิด laptop · workshop เริ่มหลัง break



# Break · 10 นาที

ยืดเส้นยืดสาย · เติมกาแฟ · เปิด laptop พร้อม Workshop

## Pre-flight Workshop

- เปิด terminal ไว้รอ
- เช็ค Claude Code: `claude --version`
- เช็ค Node.js: `node -v` ( $\geq 18$ )
- เช็ค Python 3: `python3 --version`
- Clone repo + GitLab access พร้อม

## PART 2 · WORKSHOP

# ลงมือทำ · 60 นาที

### เป้าหมาย

- ทุกคนติดตั้ง plugin สำเร็จ
- รับ skill ตัวแรกของตัวเองใน session นี้
- เปิด PR แรก (แม้แค่ typo fix ก็ได้)

### รูปแบบ

- จับคู่ 2 คน · helper-driver
- Tester Lead เดินดูช่วยทุก lab
- ติดตรงไหน ยกมือ — ไม่ต้อง Google
- มี dummy SRS + bug ตัวอย่างให้ใช้

# Lab 0 • Install Plugin (5 นาที)

ของจริงควรเตรียมมาก่อนวันงาน • ใน lab ทำอีกที + verify ให้ทุกคนพร้อมตรงกัน

## 1 Install CLI + clone repo (ถ้ายัง)

```
npm install -g @anthropic-ai/claude-code  
cd ~/Documents/GitHub  
git clone https://gitlab.ayodiacompany.com/ayodia-tester-teams/qa_ai_skill.git
```

## 2 เปิด Claude → add marketplace → install

```
claude  
> /plugin marketplace add ~/Documents/GitHub/qa_ai_skill  
> /plugin install qa-ai-skill@ayodia-qa  
> /reload-plugins
```

💡 path ใช้ ~/... ได้ Claude expand เอง • หรือใส่ absolute path เต็ม

## 3 Verify

- `/plugins` → tab Installed ต้องเห็น qa-ai-skill
- `/help` → เห็นรายการ skill (test-case-writer, bug-report-writer, ...)
- 🙋 ติดตรงไหน — ยกมือ Tester Lead เดินไปช่วย

# Lab 1 · รับ Skill ตัวแรก (5 นาที)

**Skill:** bug-report-writer · **Goal:** เขียน bug ตัวแรกผ่าน skill

## ขั้นตอน

1. เปิด Claude Code · `claude`
2. พิมพ์: `/bug-report-writer`
3. ใส่ bug ตัวอย่าง (ดูผังขวา)
4. รอ AI generate
5. อ่าน output — **Severity, Priority, Action Label** ครบมั๊ย?
6. ลอง `/bug-report-writer` อีกรอบ — ขอเป็น **EN** ดู

## Sample bug

เจอปัญหาในหน้า checkout:  
ใส่ coupon ช้อน 2 ใบ ยอดรวมผิด  
คาดว่าจะได้ส่วนลดแค่ใบเดียว  
แต่ระบบลด 2 ใบ

Browser: Chrome 130 macOS  
Env: staging  
TC: SHOP-TC-018  
Severity: Major

## Done criteria

Output มี: Title · Steps · Expected vs Actual · Env · Severity **Major** · Priority **High** · Action Label **Standard High** · Reproduce steps ครบ

# Lab 2 · เขียน TC + Tuning (10 นาที)

**Skill:** test-case-writer · **Goal:** เขียน TC + ลอง tuning per-run

## Round 1 · Default

1. ใช้ dummy SRS: examples/srs-login-sample.md
2. พิมพ์: `/test-case-writer`
3. Prompt: เขียน SIT TC จาก srs-login-sample.md ภาษาไทย
4. อ่าน output — กี่ TC? ครอบคลุม ECP/BVA มั้ย?

## Round 2 · Tune

1. ลองอันนี้ดู:  
`/test-case-writer`  
`@srs-login-sample.md`  
เน้น negative case + boundary  
+ Decision Table สำหรับ  
combo username/password  
+ State Transition login flow
1. เปรียบ output round 1 vs round 2

## ทามตัวเอง

Round 2 ครอบคลุมกว่า round 1 มั้ย? · ถ้าครอบคลุม — เก็บแค่ที่ make sense · ถ้าขาด — สั่ง tune เพิ่ม

# Lab 3 · project-context.md (5 นาที)

**Goal:** ใส่ context ของ project ใน 1 ไฟล์ → skill ใช้อัตโนมัติ ไม่ต้องบอกซ้ำ

## ขั้นตอน · 3 ข้อ

1. ใน working dir → สร้างไฟล์ project-context.md แล้ว copy template  
ฝั่งขวา
2. รับ `/test-case-writer` กับ SRS เดิม (จาก Lab 2)
3. ดู output → ควรมี **URL ของ SIT** + **คำย่อ AT** ปรากฏ (ไม่ต้องบอก skill ซ้ำ)



### Tip

เพิ่ม section อื่น (Business Rules / NFR / Severity) ได้ทีหลังเมื่อต้องใช้จริง ·  
เริ่มเล็กๆ ก่อน

## Template (เริ่มแค่ 2 section)

# **project-context.md**

### ## **Environment**

- SIT URL: <https://sit.demo.com>

### ## **Glossary**

- AT = Assessment Tax



# Lab 4 · แก้ Skill + เปิด PR (10 นาที)

Goal: ลอง full flow · แก้ skill → validate → เปิด PR

## 1 เลือกของแก้แบบเล็ก

เพิ่ม example ใน skills/bug-report-writer/examples/ · หรือเพิ่มประโยคใน trigger description

## 3 Validate

```
python3 scripts/validate_skills.py
# exit 0 = pass
```

## 5 Commit + Push (GitHub Desktop)

1. tab **Changes** → เห็นไฟล์ที่แก้
2. Summary: docs: ... + Description
3. ปุ่ม **Commit to** → **Push origin**

## 2 Branch + แก้ (GitHub Desktop)

1. GitHub Desktop → **Current Branch** → **New Branch**
2. ชื่อ: docs/bug-report-example
3. ปุ่ม **Publish branch**
4. แก้ไฟล์ตาม所选 ใน editor

## 4 Test ใน Claude

1. `/reload-plugins`
2. รัน skill อีกรอบ — ของใหม่ขึ้นมัย?

## 6 เปิด PR + ขอ review (GitLab web)

1. browser → GitLab → **Merge Requests** → **New**
2. target = master
3. ใส่ PR description ตาม template
4. ขอ Tester Lead review

# Free Play · ลองใช้เอง (15 นาที)

**Goal:** เลือก skill ที่อยากใช้จริง · ลองรันกับงานของตัวเอง · ติดตามไหน Lead เดินดูช่วย

## 💡 ไอเดียลอง

- เอา BRD/SRS งานจริงมา → /test-case-writer
- เอา bug ค้างใน Jira → /bug-report-writer
- มี TC เติม → /test-case-reviewer ภา gap
- ทดลอง /requirement-analyzer กับ spec ที่ยังไม่ชัด
- สลับ language TH/EN ดู

## 🛖 ระหว่างเล่น

- Lead เดินดูช่วยตามจุด
- สงสัยไหม → ถามได้ทันที (ไม่ต้องรอ Q&A)
- เจอ output แปลก → screenshot เก็บไว้แชร์ใน Wrap-up
- ติดยาว → ขอ pair กับเพื่อนข้างๆ

**ไม่มีถูกผิด** — เป้าหมายช่วงนี้คือให้คุ้นมือ · ของจริงเริ่มใช้ใน sprint หน้า

# ถ้าจะ Tuning · มี 3 ระดับ

## Level 1 · Per-Run

**ปรับเฉพาะครั้งนี้** — เขียน prompt เพิ่มตอนเรียก skill

ตัวอย่าง:

```
/test-case-writer
@srs.md เน้น
negative case +
boundary value
```

✅ ใช้บ่อย · ไม่ต้องแก้ไฟล์

❌ ไม่ persist

## Level 2 · Per-Project

**ปรับเฉพาะ project** — สร้าง project-context.md ใน working dir

ตัวอย่าง:

```
## Environment
SIT URL: ...
## NFR
p95 ≤ 3s
## Glossary
AT = Assess Tax
```

✅ ใช้ข้าม session · ใน project

❌ ทีมอื่นไม่ได้

## Level 3 · แก่ Skill

**แก้ SKILL.md / template** — ทุกคนในทีมได้

เคสที่ต้องแก้:

1. พู pattern ที่ AI พลาดซ้ำ
2. qa-standards เปลี่ยน
3. เพิ่ม technique ใหม่ (ECP/BVA)
4. เพิ่ม template ภาษา/format

✅ shared ทั้งทีม

⚠️ ต้องผ่าน PR review

# ถ้าจะแก้ Skill · 6 Steps

## 1 สร้าง branch (GitHub Desktop)

1. เปิด GitHub Desktop → repo qa\_ai\_skill
2. ปุ่ม **Current Branch** (ด้านบน) → **New Branch**
3. ตั้งชื่อ เช่น fix/test-case-writer-bva → **Create Branch**
4. ปุ่ม **Publish branch**

ชื่อ branch: <type>/<skill>-<short> · type = fix/feat/docs/refactor

## 3 แก้ + ทดสอบใน Claude

1. แก้ไฟล์
2. Claude session: /reload-plugins
3. ลองรัน skill กับ test input จริง
4. verify output ตรงตาม Quality Gate

## 5 Commit + Push (GitHub Desktop)

1. tab **Changes** → ตรวจไฟล์ที่จะ commit
2. ใส่ Summary + Description ช้องล่างซ้าย
3. ปุ่ม **Commit to**
4. ปุ่ม **Push origin** (toolbar บน)

## 2 หาไฟล์ที่ต้องแก้

- SKILL.md — instruction คลັก
- templates/ — output template
- examples/ — ตัวอย่าง input/output
- references/qa-standards.md — ถ้าแก้ standard ทั้งทีม

## 4 Run validate script

```
python3 scripts/validate_skills.py
```

เช็ค frontmatter / 8 sections / dead links / deprecated codes

## 6 เปิด PR (GitLab web)

1. เปิด browser → GitLab repo
2. menu **Merge Requests** → **New**
3. fill template (ดู slide ถัดไป)
4. request review **Tester Lead** + 1 peer

# Pre-flight Checklist · ก่อนเถ้า

## 📖 อ่านก่อน

1. `SKILL-TEMPLATE.md` — universal 8-section
2. `references/ai-guardrails.md` — 5 คลัง guardrail
3. `references/qa-standards.md` — Severity/Priority/Sizing
4. `references/sdp-mapping.md` — process → skill mapping
5. SKILL.md ของ skill ที่จะเถ้า

## 🔧 Tool ที่ต้องมี

1. Claude Code CLI ( `npm i -g @anthropic-ai/claude-code` )
2. Python 3 ( `validate_skills.py` )
3. GitHub Desktop + GitLab access (sign in คำ remote ของ repo)

## 🤖 ถามตัวเองก่อน

### Q1 · ปัญหาเกิดซ้ำมั๊ย?

ถ้าครั้งเดียว → Level 1 (per-run prompt) พอ  
ถ้าซ้ำ  $\geq 3$  ครั้ง → Level 3 (เถ้า skill)

### Q2 · กระทบ skill อื่นมั๊ย?

ถ้าเถ้า Severity scale → กระทบทุก skill ที่ใช้  
ต้องเถ้าที่ qa-standards.md ไม่ใช่เถ้าทีละ skill

### Q3 · กระทบ user ปัจจุบันมั๊ย?

breaking change → ต้อง flag ใน PR  
backward-compat → ใส่ ติดท้าย safe

# Validate ก่อน Commit

ทุก PR ผ่าน CI · แต่รัน local ก่อนช่วยให้ feedback loop เร็วขึ้น

## Auto check

`python3 scripts/validate_skills.py`

Script ใช้ 5 อย่าง:

1. Frontmatter คสว + name = folder name + unique
2. 8 sections มาคสว + ลำดับถูกต้อง
3. Link ไป references/ai-guardrails.md
4. ไม่ใช้ deprecated code (P0-P3, Sev1-4)
5. Markdown link ไม่ dead

exit 0 = pass · exit 1 = fail (CI fail)

## Manual test

1. เปิด Claude Code: `claude`
2. `/reload-plugins` → โหลด skill version ใหม่
3. รัน skill กับ test input **จริง** (ใช้ examples/)
4. Verify output ตาม Quality Gate ใน SKILL.md
5. ลอง edge case ที่เป็นเหตุให้เกิดตั้งแต่วแรก

### ห้ามลืม

ถ้าแก้ template → ต้อง regenerate ตัวอย่างใน examples/ ด้วย เพื่อให้ user คนต่อไปเห็น output แบบใหม่

# วิธีเปิด PR · ผ่าน GitHub Desktop + GitLab web

**Note:** repo อยู่บน GitLab → “PR” ในที่นี้ = GitLab Merge Request · GitHub Desktop ใช้ commit/push เสร็จแล้วเปิด PR ใน GitLab web เอง

## 1 Commit + Push ใน GitHub Desktop

- tab **Changes** → เห็นไฟล์ที่แก้
- ใส commit message ช้องล่าง (Summary + Description)
- กดปุ่ม **Commit to**
- กดปุ่ม **Push origin** (toolbar ด้านบน)

## 2 เปิด GitLab web → New Merge Request

- เปิด browser ไป [gitlab.ayodiacompany.com/.../qa\\_ai\\_skill](https://gitlab.ayodiacompany.com/.../qa_ai_skill)
- menu **Merge Requests** → **New merge request**
- Source = branch ของเรา · Target = master

## 3 เขียน Title + Description

Title format: <type>: <skill> – <change ย่อ>

เช่น: fix: test-case-writer – add BVA reminder for date fields

## 4 Assign reviewer + รอ CI

**Required:** Tester Lead + 1 peer · CI pipeline ต้องเขียว ก่อน merge

# PR Description Template

Copy template นี้ใส่ใน PR description (GitLab → Merge Request description box) — กรอกตามหัวข้อ

## ## What

แก้ไขอะไร – 1-2 บรรทัด

## ## Why

ทำไมต้องแก้ – link Jira ticket / ตัวอย่าง output ที่พลาด / pattern ที่ซ้ำ

## ## How

- เปลี่ยนใน SKILL.md section X
- เพิ่ม template Y
- update qa-standards.md (ถ้ากระทบ standard)

## ## Test

- [ ] รัน python3 scripts/validate\_skills.py ผ่าน
- [ ] ทดสอบ skill กับ input จริง: <ตัวอย่าง / paste output>
- [ ] ตรวจ Quality Gate ใน SKILL.md ครบ
- [ ] ตรวจกระทบ skill อื่นที่เกี่ยวข้อง

## ## Breaking change?

- [ ] No
- [ ] Yes → notify ทีม + อัปเดต CHANGELOG

## ## Sample output

<paste output ตัวอย่างหลังแก้>



# DOs · DON'Ts ตอน Tuning

## ✓ DOs

- แก้ที่ root cause · ถ้า standard ผิด → แก้ qa-standards.md ไม่แก้ที่ละ skill
- เพิ่ม example ทุกครั้งที่เพิ่ม technique ใหม่
- test กับ input จริง 2-3 cases ก่อนเปิด PR
- เขียน description ละเอียด (Why > What)
- ขอ peer review ตั้งแต่ draft PR
- update examples/ พร้อม template
- run validate\_skills.py ก่อน push

## ✗ DON'Ts

- ห้าม hardcode company-specific (URL/IP/DB) ใน SKILL.md
- ห้าม commit credential / token / PII (ใช้ [REDACTED])
- ห้าม push ตรงเข้า master — ทุกอย่างผ่าน PR
- ห้ามแก้ Severity/Priority scale โดยไม่ update qa-standards
- ห้าม merge เอง — ต้องรอ Tester Lead approve
- ห้าม skip CI · ห้าม --no-verify
- ห้าม merge ระหว่าง CI ยัง red

# Versioning · ติดตามการแก้ไข

## 📖 ดู history ของ skill

# log ของ skill ใดๆ

```
git log --oneline skills/test-case-writer/
```

# ดู diff รอบล่าสุด

```
git log -p -1 skills/test-case-writer/SKILL.md
```

# ใครแก้บรรทัดไหน

```
git blame skills/test-case-writer/SKILL.md
```

ทุกการแก้ผ่าน PR → อยู่ใน git history → reverse engineer ได้ตลอด

## 🔔 รู้ได้ใจว่ามีของใหม่

1. Watch repo ใน GitLab — get notify ทุก PR (Merge Request)
2. อ่าน CHANGELOG.md (เพิ่มเข้า repo เร็วๆ นี้)
3. Weekly Update email — Tester Lead สรุป skill ที่อัปเดต
4. `git pull` + `/plugin marketplace update ayodia-qa` ทุกต้นสัปดาห์

### 💡 Tip

ถ้าแก้ skill แล้ว breaking — โฟลว์ Slack Tester channel + tag ทุกคนที่ใช้ skill นั้นบ่อย

# | สิ่งที่ยากให้ทำต่อ

## ● อาทิตย์นี้

1. ติดตั้ง plugin ตาม README (5 นาที)
2. ลองรัน 1 skill ที่ใช้บ่อยสุด — bug-report-writer หรือ test-case-writer
3. อ่าน docs/qa-onboarding.md (15 นาที)
4. บันทึก feedback: ขงง่าย / ยาก / ขาดอะไร

## ● อาทิตย์ถัดไป

1. ใช้ skill ใน sprint จริง — เก็บตัวเลข effort saving
2. เจอจุดที่ output ไม่ตรง standard → แจ้งใน Slack
3. ถ้าอยากเพิ่ม feature เล็กๆ → ลองเปิด PR แรก (docs: หรือ fix:)
4. Pair กับ Tester Lead ทำ PR ตัวแรกของตัวเอง

**เป้าหมาย sprint หน้า:** ทุกคนเปิด PR แรกของตัวเอง · จุดเริ่มที่ดี = แก้ typo / เพิ่ม example / ปรับ wording

# Q & A

## อ่านต่อ

1. `README.md` — ภาพรวม + install
2. `docs/qa-onboarding.md` — Quick Start 5 นาที + Decision Tree
3. `docs/how-to-sit-uat.md` — 10 steps พิมพ์อะไรให้ AI ทุกขั้น
4. `SKILL-TEMPLATE.md` — universal 8-section
5. `references/qa-standards.md` — Severity/Priority/Sizing/KPI
6. `references/ai-guardrails.md` — 5 หลักการ

ติดปัญหา / มีไอเดียเพิ่ม skill — ทักใน Slack #qa-ai-skill หรือ Tester Lead Team